

Assignment 2: Model to Web Application

AI Spreadsheets and Python Programming

Assignment Objective

Each student/team will work with one assigned management-domain dataset. The dummy dataset will be generated later using Copilot from the provided column specification. The goal is to move from raw business data to a statistically sound linear regression model and then integrate the trained model into a simple interactive web application.

Dataset Requirement

- Use the dataset assigned from the separate Dataset Specifications document.
- Generate a realistic dataset with sufficient rows using Copilot. Keep the exact column names given in the specification.
- The target variable is the final numeric column listed for the assigned dataset unless specified otherwise.
- Do not begin model building before checking data quality and the assumptions required for linear regression.

Part A - Data Preparation and Linear Regression Assumptions

Perform exploratory checks, diagnose the assumptions of multiple linear regression, and apply appropriate corrective steps where required. Your notebook/report should show both the diagnosis and the action taken.

- 1. Data quality:** Check missing values, duplicates, impossible values, inconsistent categories, and obvious data-entry issues. Treat them appropriately.
- 2. Target and feature understanding:** Identify the dependent variable, numerical predictors, categorical predictors, and any identifier columns that should not be used directly as predictors.
- 3. Linearity:** Examine whether the relationship between the target and continuous predictors is reasonably linear using plots and/or residual diagnostics. Apply suitable transformations or feature engineering when justified.
- 4. Multicollinearity:** Check correlations among predictors and calculate VIF or an equivalent diagnostic. Remove, combine, or redesign strongly redundant predictors when required.
- 5. Independence of errors:** Inspect residual behavior and use a suitable diagnostic such as Durbin-Watson when relevant. Explain whether independence is a reasonable assumption for the generated data.
- 6. Homoscedasticity:** Use residual-versus-fitted plots and, where appropriate, a formal test such as Breusch-Pagan. If heteroscedasticity appears, consider transformations, robust treatment, or redesign of variables.
- 7. Normality of residuals:** Inspect histogram/Q-Q plot of residuals and optionally use a formal test. Focus on residuals rather than assuming every input feature must be normally distributed.
- 8. Outliers and influential observations:** Use standardized residuals, leverage, Cook's distance, IQR or related methods. Investigate influential points before deciding whether to retain, cap, transform, or remove them.

9. Categorical-variable treatment: Encode categorical predictors appropriately. Avoid accidental ordinal meaning when categories are nominal.

10. Re-check assumptions: After corrections, re-fit the regression and show that the major assumption violations have been reduced or explain any remaining limitations.

Part B - Build and Evaluate the Machine Learning Model

- Create a reproducible preprocessing and modelling workflow in Python.
- Split the data into training and testing sets. Clearly state the split used.
- Build a multiple linear regression model as the primary model.
- Report model coefficients and explain the business meaning of the most important predictors.
- Evaluate the model using R-squared, Adjusted R-squared where appropriate, MAE, MSE/RMSE, and residual analysis.
- Optionally compare the linear model with one additional regression algorithm, but the linear regression analysis remains compulsory.
- Use a preprocessing + model Pipeline whenever practical so that the same transformations are applied during prediction.

Part C - Save the Trained Model

- Save the final trained preprocessing/model pipeline as a .pkl file using pickle or joblib.
- Keep feature names, preprocessing steps, and expected input formats consistent between the training notebook and the application.

Part D - Vibe-Code a Web Application

Develop a small user-facing prediction application that loads the saved model and predicts the numeric business outcome. Streamlit is encouraged because it supports rapid prototyping, but another simple web framework may be used.

- Provide input controls for the predictor variables required by the model.
- Load the saved .pkl model inside the application rather than re-training the model every time the app starts.
- Display the predicted value clearly with the correct business unit such as revenue, units, premium, hours, or profit.
- Add basic validation so obviously invalid inputs are not accepted silently.
- Include a short explanation of what the prediction means for a manager or product owner.
- The visual design can be created through vibe coding, but the underlying model workflow and input handling must be correct.

Suggested Application Files

File	Purpose
data.csv	Copilot-generated dataset used for analysis and training
model_training.ipynb	EDA, assumption checks, corrections, model training and evaluation
model.pkl	Saved final preprocessing/model pipeline
app.py	Streamlit or other web application
README.md	How to run the project and short business problem description

Submission Checklist

- Generated dataset file
- Completed Jupyter notebook with assumption checks and fixes
- Saved model.pkl file
- Working web application source code
- Short README with run instructions and screenshots of the working application